# DevDoc Studio — SRS status report

Where the application stands against the 52 requirements in the Software Requirements Specification, what remains, and a recommended order to build it in. Each requirement was checked against the current code; the companion spreadsheet *SRS-DevdocStudio-Gap-Analysis.xlsx* carries the same status and notes per row.

**52** REQUIREMENTS ASSESSED

**12** DONE

**19** PARTIALLY DONE

**21** NOT STARTED

| AREA | PROGRESS | DONE | PARTIAL | NOT STARTED | TOTAL | % DONE |
|---|---|---|---|---|---|---|
| Document editor & UI | 8 / 2 / 1 | 8 | 2 | 1 | 11 | **73%** |
| Test manager | 4 / 11 / 6 | 4 | 11 | 6 | 21 | **19%** |
| Release management | 4 / 10 | 0 | 4 | 10 | 14 | **0%** |
| Non-functional | 2 / 4 | 0 | 2 | 4 | 6 | **0%** |
| **All requirements** | 12 / 19 / 21 | 12 | 19 | 21 | 52 | **23%** |

## Key points

1. **Document editing is essentially complete** (8 of 11 done): permissions, dashboard, editor, VS Code export and download all work, enforced by the backend. One real access gap: data management (incl. “clear all data”) is open to every user (008).
2. **Test management has a solid base but is browser-only.** Suites, runs, manual statuses and AI test generation work, but data is stored per browser, so it is not shared between users. Moving it to the backend unlocks most of the remaining test-manager items.
3. **Remote execution runs against a mock** Test PC. The push → start → poll flow exists end to end; a real server registry, backend worker and live updates are still to build.
4. **Release management is the largest gap** (0 of 14 done): the Release Master screen displays sample data only.
5. **Recently completed:** the backend was migrated to Python/FastAPI with an identical API, the repository restructured into frontend/backend, CI extended to the Python backend, and dark-theme readability fixes.

# Recommended order of work

Every requirement that is not yet done appears exactly once below. The order follows dependencies: fix what is broken and cheap first, then build the backend foundation that test execution and release management both rely on. Effort is relative: S = days, M = 1–2 weeks, L = several weeks.

| PHASE | FOCUS | REQUIREMENTS (COLOUR = CURRENT STATUS) | WHY IN THIS ORDER |
|---|---|---|---|
| 0 | **Quick wins & fixes**<br>Effort: S | 008 011 007 013 018 015 024 | Small, self-contained changes on screens that already exist: wire up the admin-only data management check (a real access gap today), make the audit log admin-only, add a Data Management sidebar entry, complete the test case fields, attach issue IDs to failed results, add CSV/JSON import-export and CSV/PDF run reports. |
| 1 | **Move test management to the backend**<br>Effort: L | 012 014 073 031 025 | Test suites, cases and runs live in each user's browser today, so nothing is shared, searchable or auditable. Moving them into Postgres behind the API is the foundation most later items need: Projects and sub-sections, revision history, the 5,000-case performance target, consolidated reports and run-to-run trends. |
| 2 | **Real test execution engine**<br>Effort: L | 019 075 020 021 022 023 072 028 026 027 | Replace the mock Test PC: a server registry with encrypted credentials, a backend polling worker with configurable intervals and full job states, live push updates to the UI, metrics, then scheduled runs and Jenkins/Bitbucket triggers. Needs Phase 1 (runs must be stored server-side) and the hardware API details. |
| 3 | **Release management**<br>Effort: L | 037 038 039 040 035 034 042 043 044 045 041 033 046 036 | Today Release Master only displays sample data. Build it in dependency order: create releases and milestones, register RC builds and their artifacts, link test runs to RCs, evaluate gates from real run results and Jira blockers, collect multi-role sign-offs, then promotion to Final, overrides and an immutable audit log. |
| 4 | **Security & platform hardening**<br>Effort: M | 074 076 071 030 | HTTPS/TLS 1.3 for the deployment, SSO/OAuth2 sign-in, a measured performance baseline, and connecting existing repositories from the UI. TLS can be pulled earlier as soon as a real deployment target exists. |

Chip colours: `partial` `not started`. Phases 2 and 3 can run in parallel once Phase 1 is in place, if two people are available.

# Document Editor & UI

DDS-SRS-001 – 011 · 8 done · 2 partial · 1 not started

| # | REQ ID | REQUIREMENT | STATUS | WHAT EXISTS TODAY / WHAT IS MISSING |
|---|---|---|---|---|
| 1 | DDS-SRS-001 | **List in-progress documents; open in editor** | Done | Documents list shows in-progress (Draft / In Review) documents; select to open in the editor. Enforced server-side too (GET /api/documents). |
| 2 | DDS-SRS-002 | **Users edit only their own documents** | Done | Non-admins can only edit/delete documents they authored (permissions.ts; re-checked by the API: 403 otherwise). |
| 3 | DDS-SRS-003 | **Admin views all documents** | Done | Admins see every document, any status, and can open it in the editor. |
| 4 | DDS-SRS-004 | **Admin edits all documents** | Done | Admins can edit and delete all documents (frontend + API). |
| 5 | DDS-SRS-005 | **Dashboard landing page with document stats** | Done | Dashboard is the landing page after login: total / approved / in-review / draft counts and latest Google Drive upload status. |
| 6 | DDS-SRS-006 | **Sidebar entry for the document editor** | Done | Sidebar item 'Document Editor'. |
| 7 | DDS-SRS-007 | **Sidebar: doc editor, data management, code editor** | Partial | Sidebar has Document Editor and Coder (code editor). No separate 'Data management' item - data/backup tools sit under Settings > System & Storage. |
| 8 | DDS-SRS-008 | **Data management restricted to admins** | Not started | An admin-only check exists (canAccessDataManagement) but is not wired up: every user can open Settings > System & Storage, including 'clear all data'. |
| 9 | DDS-SRS-009 | **Save document to the VS Code project folder** | Done | Export to VS Code saves the document (.md, with .docx) into the Coder workspace's specs/docs folders via the file bridge; falls back to browser download if the workspace is offline. |
| 10 | DDS-SRS-010 | **Download documents** | Done | Download as .docx (plus .md / JSON); Google Docs export also available. |
| 11 | DDS-SRS-011 | **Admin audit log of user sessions** | Partial | Settings shows a User Session Audit Log (user, role, login, logout, duration). Not admin-only, and it is stored per browser, so it only reflects sessions on that machine (other users are sample data). |

# Test Manager

DDS-SRS-012 – 032 · 4 done · 11 partial · 6 not started

| # | REQ ID | REQUIREMENT | STATUS | WHAT EXISTS TODAY / WHAT IS MISSING |
|---|---|---|---|---|
| 12 | DDS-SRS-012 | **Hierarchy: Projects > Suites > Sections > Cases** | Partial | Suites → Sections → Test Cases exist. No 'Project' level; the data model allows sub-sections (parentId) but the UI does not create or show them. |
| 13 | DDS-SRS-013 | **Test case fields (ID, steps, post-conditions, attachments)** | Partial | Has auto-generated ID (time-based 'C1234', not sequential 'TC-104'), title, pre-conditions, type, priority (P1-P4 rather than Low..Critical), steps with expected result, one evidence screenshot. Missing: post-conditions, log-file attachments, multiple attachments. |
| 14 | DDS-SRS-014 | **Revision history for test cases** | Not started | No revision history for test cases (version history exists for documents only). |
| 15 | DDS-SRS-015 | **Import/export test cases as CSV, XML, JSON** | Partial | Import/export via Excel (.xlsx); export as Markdown and .robot; import Robot output.xml results; export results JSON. Not CSV, XML or JSON import/export of test cases themselves. |
| 16 | DDS-SRS-016 | **Create test runs from cases or whole suites** | Done | Add Test Run: include all cases of a suite, select specific cases, or dynamic filtering. |
| 17 | DDS-SRS-017 | **Manual execution statuses** | Done | Per-case status in a run: Passed, Failed, Blocked, Retest, Untested, Skipped. |
| 18 | DDS-SRS-018 | **Attach Jira/GitHub issue IDs to failed results** | Partial | Test cases carry Jira references and a run's Defects tab lists failed cases, but an issue ID cannot be attached to an individual failed result. |
| 19 | DDS-SRS-019 | **Register remote test servers (URL, auth, payload)** | Not started | No server registry: the app talks to one backend set at deploy time (VITE_API_BASE_URL). No UI for endpoint URL, auth credentials or payload parameters. |
| 20 | DDS-SRS-020 | **Trigger remote runs from the UI via REST** | Partial | 'Run Test' pushes the suite's Robot script and starts a run via REST - against a mock remote Test PC (randomised results); no real hardware integration yet. |
| 21 | DDS-SRS-021 | **Backend polling service (interval, states, metrics)** | Partial | Polling exists but runs in the browser, every 30 min (fixed), with only RUNNING/COMPLETED states and pass/fail counts. Needs: backend worker, configurable 5/10/30 s interval, QUEUED/FAILED states, completion %. |
| 22 | DDS-SRS-022 | **Real-time UI updates without refresh** | Not started | No push updates (no WebSocket/SSE); results appear on the next poll. |

# Test Manager (continued)

DDS-SRS-012 – 032 · 4 done · 11 partial · 6 not started

| # | REQ ID | REQUIREMENT | STATUS | WHAT EXISTS TODAY / WHAT IS MISSING |
|---|---|---|---|---|
| 23 | DDS-SRS-023 | **Metrics dashboard (completion %, donut, velocity)** | Partial | Run view has a pass/fail donut chart and pass percentage. No velocity/trend graphs. |
| 24 | DDS-SRS-024 | **Downloadable summary reports (PDF, HTML, CSV)** | Partial | Run report exports to Excel (.xlsx); results JSON export; sample Robot HTML report. No PDF or CSV summary report or template. |
| 25 | DDS-SRS-025 | **Compare runs to detect regressions** | Not started | No comparison of past runs vs current run / regression detection. |
| 26 | DDS-SRS-026 | **Run tests locally and in CI/CD** | Partial | Robot Framework can be run locally in the Coder sandbox (VS Code container); no CI/CD execution path. |
| 27 | DDS-SRS-027 | **Jenkins and Bitbucket integration** | Not started | No Jenkins or Bitbucket integration. |
| 28 | DDS-SRS-028 | **Scheduled runs** | Not started | No scheduled runs. |
| 29 | DDS-SRS-029 | **Integrated code editor with AI generation** | Done | Coder screen embeds VS Code (code-server) with the Cline AI agent; Test Management generates test cases and Robot scripts from a Technical Analysis document with AI (Gemini/OpenAI/Ollama). |
| 30 | DDS-SRS-030 | **Connect existing code** | Partial | The Coder workspace can hold existing code (mounted folder / git clone in its terminal); no in-app 'connect repository' option. |
| 31 | DDS-SRS-031 | **Consolidated manual + automated report** | Partial | Runs record manual and automated counts together, but there is no single consolidated manual + automated report export. |
| 32 | DDS-SRS-032 | **Share docs/test cases with the AI code editor** | Done | Documents, specs and test suites can be pushed into the AI code editor's workspace (file bridge) for test/script generation. |

# Release Management

DDS-SRS-033 – 046 · 0 done · 4 partial · 10 not started

| # | REQ ID | REQUIREMENT | STATUS | WHAT EXISTS TODAY / WHAT IS MISSING |
|---|---|---|---|---|
| 33 | DDS-SRS-033 | **Create releases, approve RC>Final, override gates, sign off** | Not started | Release Master is view-only: releases cannot be created, RC→Final approved, gates overridden or signed off in-app. |
| 34 | DDS-SRS-034 | **Link releases to test runs; log blocking defects** | Not started | Releases link to a Technical Analysis doc and a Test Report doc, not to Test Runs; no defect logging on releases. |
| 35 | DDS-SRS-035 | **Register build artifacts (checksums, URLs)** | Not started | No build-artifact registration. |
| 36 | DDS-SRS-036 | **View release status, metrics and release notes** | Partial | Shows release status, story-point burndown, readiness and documentation status. No release notes. |
| 37 | DDS-SRS-037 | **Create a release with version, type, date, status** | Not started | Cannot create releases. Existing releases (sample data) have version, target date and status (Planning / RC Testing / Approved for GA / Deployed) but no type or customer segment. |
| 38 | DDS-SRS-038 | **Release phase milestones (freeze ... GA)** | Not started | No release phase milestones (Test Management has test milestones, not release phases). |
| 39 | DDS-SRS-039 | **Multiple RC builds per release** | Not started | No Release Candidate builds per release. |
| 40 | DDS-SRS-040 | **Image details per build (tag, SHA, URL, checksum)** | Not started | No image tracking (tag, commit SHA, URL, SHA-256, build ID). |
| 41 | DDS-SRS-041 | **Promote an RC image to Final** | Not started | No RC → Final promotion. |
| 42 | DDS-SRS-042 | **Link test runs to RC images** | Not started | Test Runs cannot be linked to RC images. |
| 43 | DDS-SRS-043 | **Automatic release-readiness gates** | Partial | A Quality Gate verdict (GO / Pending QA / NO-GO) is shown, but it is a stored value, not evaluated from pass rate, open blockers or mandatory suites. |
| 44 | DDS-SRS-044 | **Link Jira/GitHub release blockers** | Partial | Each release lists its Jira tickets, but from sample data - the 'Sync JIRA' button is simulated; no blocker flag. |
| 45 | DDS-SRS-045 | **Multi-role electronic sign-offs** | Partial | Multi-reviewer sign-off exists for documents (Reviewers Log). Releases have a single sign-off owner; no multi-role release sign-off. |
| 46 | DDS-SRS-046 | **Immutable audit log for release actions** | Not started | No audit log for promotions, status changes, gate overrides or sign-offs. |

# Non-functional requirements

DDS-SRS-071 – 076 · 0 done · 2 partial · 4 not started

| # | REQ ID | REQUIREMENT | STATUS | WHAT EXISTS TODAY / WHAT IS MISSING |
|---|---|---|---|---|
| 47 | DDS-SRS-071 | **UI navigation under 1.5 s** | Partial | Single-page app with code-splitting; navigation is client-side and near-instant in practice. Not formally measured. |
| 48 | DDS-SRS-072 | **Polling supports 100 concurrent runs** | Not started | No backend polling service yet (mock runs are in memory); not load-tested. |
| 49 | DDS-SRS-073 | **5,000+ case suites load in under 500 ms** | Not started | Test cases are stored in the browser (localStorage), not the database, so there is no indexed query path to measure. |
| 50 | DDS-SRS-074 | **HTTPS / TLS 1.3 in transit** | Not started | Served over plain HTTP; TLS 1.3 is a deployment step (reverse proxy) not yet done. |
| 51 | DDS-SRS-075 | **Server API keys encrypted at rest (AES-256)** | Not started | No external test server keys stored yet (see 019). Server-side AI keys live in environment variables, not encrypted at rest. |
| 52 | DDS-SRS-076 | **JWT and OAuth2/SSO authentication** | Partial | Real login with server-side sessions (httpOnly cookie, bcrypt, rate-limited) - deliberately not JWT so logout revokes instantly. Google OAuth is used for Drive export, not for sign-in; no SSO. |

## Decisions needed to proceed

| # | DECISION | WHY IT IS NEEDED |
|---|---|---|
| 1 | **Background job queue** | The polling worker (021, 072) needs a queue: arq, Celery or RQ. Recommendation: arq (async, fits FastAPI, Redis only). |
| 2 | **Test PC / hardware API** | Endpoint, auth and result format of the real remote test servers, to replace the mock (019-021). |
| 3 | **Jira, Jenkins, Bitbucket access** | Instances and service credentials for 018, 027 and 044; current Jira sync is simulated. |
| 4 | **SSO provider** | Which identity provider for 076 (e.g. Azure AD / Okta / Google Workspace). |
| 5 | **JWT vs. sessions** | 076 asks for JWT. Login currently uses server-side sessions on purpose: logout and revocation take effect instantly. Confirm whether JWT is a hard requirement. |
| 6 | **Priority scale** | 013 specifies Low / Medium / High / Critical; the tool uses P1-P4. Keep, rename, or map? |
| 7 | **Deployment target** | Where the app will run, for TLS 1.3 (074) and the performance targets (071-073). |